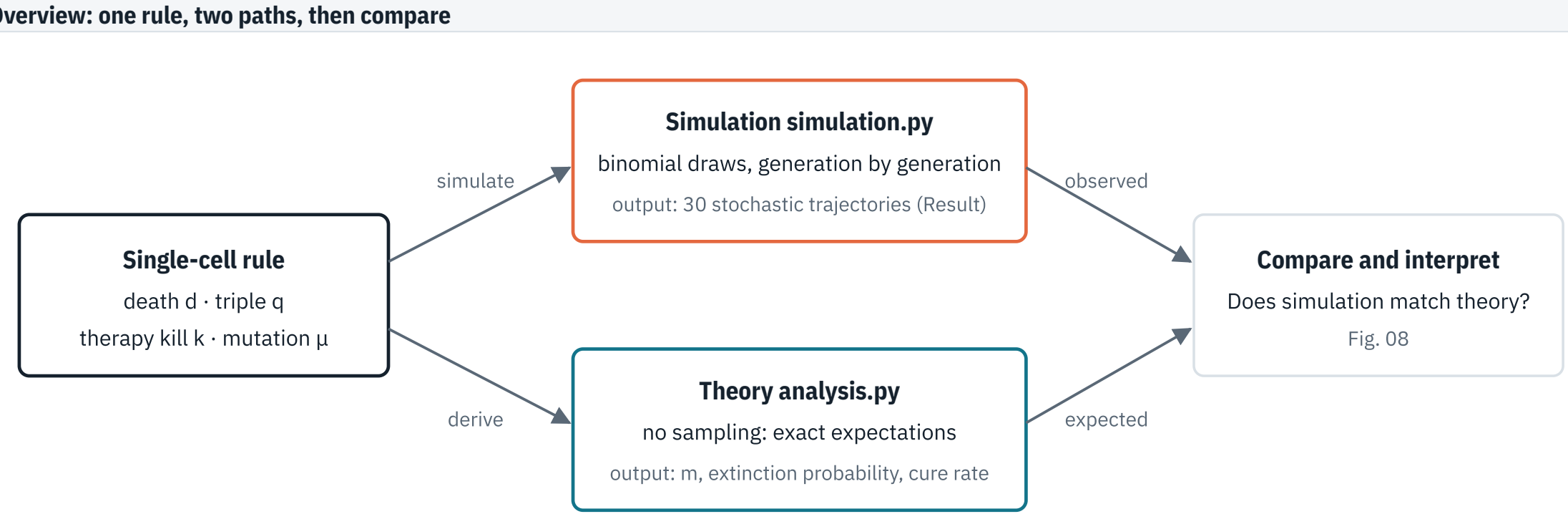
BME 6120 · QUANTITATIVE TUMOR MODELING

Tumor branching model: simulation and analysis

Code tour: how the simulation works and what the analyses do

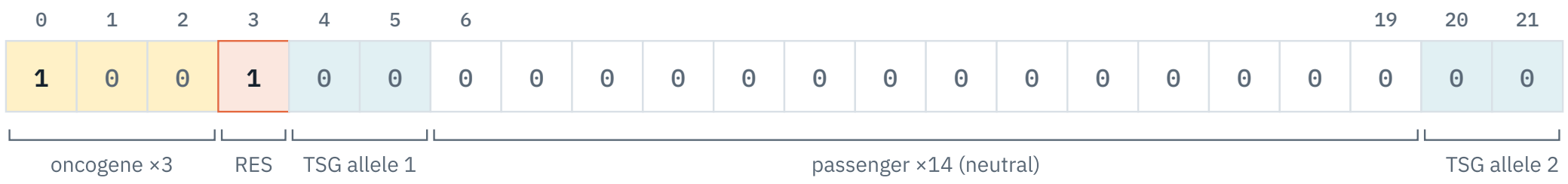
The program does two things: it uses one single-cell rule to **simulate tumors stochastically** (simulation.py), and it uses **the same rule to derive theory** that checks and interprets the simulation results (analysis.py). The other files only repeat runs, save data and draw figures.

1 Overview: one rule, two paths, then compare



The simulation and the theory use the same d, q, k and μ, so agreement means both the code and the reasoning are right; disagreement points to randomness or to genotype information the theory ignores.

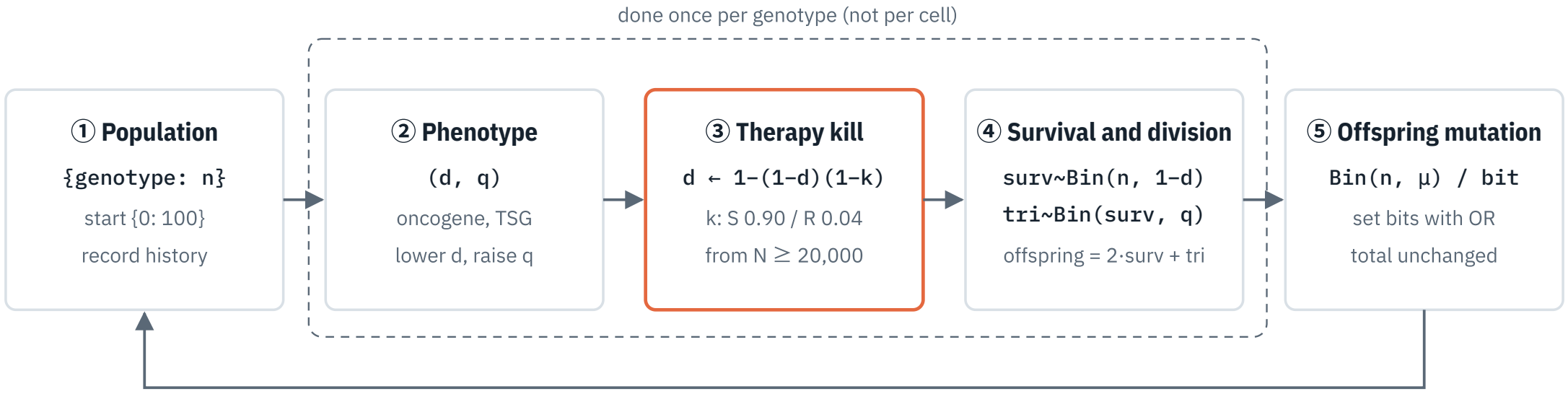
2 How the simulation represents a cell: the genotype is a 22-bit integer



Example: genotype = 9 → genes 0 and 3 are mutated (ONC1 + RES); mutating gene i = `genotype | (1 << i)`

A tumor is not a list of cell objects but a dictionary `{genotype: cell count}`. Cells with the same genotype share the same fate probabilities, so they can be sampled as one group; mutation only moves cells from one entry to another, and the total stays the same.

3 One simulated generation: five steps, repeated



Three design choices: (1) count cells grouped by genotype; (2) two binomial draws have exactly the same distribution as flipping a coin for every cell, just much faster; (3) selection first, then division, then mutation, so a new mutation takes effect from the next generation. Therapy does not change μ; resistance can only arise beforehand, by chance.

Code architecture

> **One cell-level rule drives both the simulation and the theory, so the theory is an independent check on the code.**

Model and code: one cell, one generation

Single-cell rule: discrete-time stochastic branching process

CONCEPT

Each cell independently leaves 0, 2 or 3 daughters per generation, with probabilities set only by (*d*, *q*, *k*) ⇒ a discrete-time stochastic branching process; the offspring distribution is fully described by *f*(*z*) (mutation is ignored here).

d: probability of dying in a generation (leaves 0 daughters)
q: probability that a surviving cell makes 3 daughters (otherwise 2)
k: extra kill probability from therapy (sensitive 0.90, resistant 0.04)

KEY MATH

$$P(0) = d, \quad P(2) = (1-d)(1-q), \quad P(3) = (1-d)\,q$$

P(n) = probability that one cell leaves n daughters after one generation (death = 0); a cell never leaves exactly 1

$$f(z) = d + (1-d)\left[(1-q)z^2 + q\,z^3\right]$$

probability generating function: the coefficient of z^j = probability of leaving j daughters

$$m = f'(1) = (1-d)(1-k)(2+q)$$

mean offspring; $E[N_t] = N_0\, m^t$

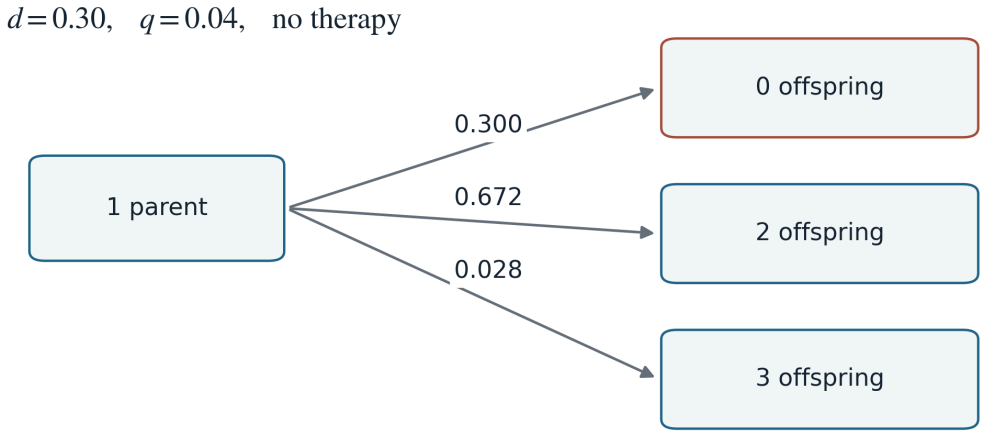
$$s = f(s) \;\Rightarrow\; s = \min\{x \in [0,1] : x = f(x)\}$$

fixed point s: probability that one cell's lineage eventually dies out; iterating s ← f(s) from s = 0 converges to the smallest root. If m > 1 then s < 1; if m ≤ 1 then s = 1.

Single-cell rules: what one cell does in one generation

A Offspring distribution of a mutation-free cell

$d = 0.30, \quad q = 0.04, \quad$ no therapy



$$f(z) = 0.300 + 0.672z^2 + 0.028z^3$$

$$m = f'(1) = 1.428, \qquad s = f(s) \Rightarrow s = 0.421$$

B What the symbols mean

d	death probability per generation (cell leaves no offspring)
q	probability that a surviving cell makes 3 daughters (else 2)
k	extra kill probability from therapy (sensitive 0.90, resistant 0.04)
$P(n)$	probability that one parent leaves n offspring
$f(z)$	offspring generating function; coefficient of z^n = P(n)
m	mean offspring per cell = (1−d)(1−k)(2+q); m > 1 grows
s	probability that one cell's lineage eventually dies out

C Numbers that decide the outcome (default parameters)

Parent phenotype	d death prob.	q P(3 daughters | survives)	P(0) no offspring	P(2) 2 daughters	P(3) 3 daughters	m, no therapy mean offspring	m, on therapy mean offspring
Baseline sensitive	0.30	0.04	0.300	0.672	0.028	**1.4280**	**0.1428**
Oncogene sensitive	0.22	0.16	0.220	0.655	0.125	**1.6848**	**0.1685**
Oncogene + TSG loss	0.14	0.28	0.140	0.619	0.241	**1.9608**	**0.1961**
Baseline resistant	0.30	0.04	0.300	0.672	0.028	**1.4280**	**1.3709**

Fig. 06 Single-cell rules

Genotype → phenotype: oncogenes and TSG loss each lower death and raise triple division

simulation.py:151–170

```python
def growth_probabilities(genotype: int, config: Config) -> tuple[float, float]:
    """Oncogene and TSG-loss advantages; they add."""
    oncogene = driver_count(genotype, config) >= config.driver_hits
    suppressor = suppressor_loss(genotype, config)
    # Each class counts once; TSGs do not stack.
    death = max(
        0,
        config.death_rate
        - config.driver_death_reduction * oncogene
        - config.suppressor_death_reduction * suppressor,
    )
    # Optional resistance cost (default 0).
    if config.resistance_cost and genotype & (1 << config.resistance_gene):
        death = min(1.0, death + config.resistance_cost)
    triple = (
        config.triple_probability
        + config.driver_triple_bonus * oncogene
        + config.suppressor_triple_bonus * suppressor
    )
    return death, triple
```

KEY POINTS

> **Genotype maps to just two numbers, death d and triple-division probability q.**

> **Each class of driver counts once, and the effects add.**

One-generation update: therapy kill → binomial survival and triples → offspring mutation

simulation.py:253–339

```python
def simulate(
    config: Config = Config(),
    seed: int = 42,
    therapy: bool = True,
    target_size: int | None = None,
) -> Result:
    """Synchronous-generation branching process; therapy stays on once started."""
    # Validate settings; create the RNG.
    config.validate()
    if target_size is not None and target_size <= 0:
        raise ValueError("target_size must be positive")
    # Fixed seed; genotype 0 = no mutations.
    rng = np.random.default_rng(seed)
    population = {0: config.initial_cells}
    # Per-generation summaries and genotype snapshots.
    history, populations = [], []
    therapy_generation = None
    therapy_on = False
    active = []
    stop_reason = "generation_limit"
    # Record the current generation, then advance.
    for generation in range(config.generations + 1):
        row = summarize(population, generation, config)
        history.append(row)
        populations.append(dict(population))
        # Stop on extinction, target size, cell cap or time limit.
        if row["total"] == 0:
            stop_reason = "extinction"
            break
        if target_size is not None and row["total"] >= target_size:
            stop_reason = "target_reached"
            break
        if row["total"] >= config.max_cells:
            stop_reason = "cell_limit"
            break
        if generation == config.generations:
            break
        # Therapy starts at the first threshold crossing and stays on.
        if (
            therapy
            and therapy_generation is None
            and row["total"] >= config.therapy_start_size
        ):
            therapy_generation = generation
            therapy_on = True
        # adaptive: pause when burden falls, resume at threshold.
        elif therapy_generation is not None and config.therapy_mode == "adaptive":
            reference = history[therapy_generation]["total"]
            if (
                therapy_on
                and row["total"] <= config.adaptive_pause_fraction * reference
            ):
                therapy_on = False
            elif not therapy_on and row["total"] >= config.therapy_start_size:
                therapy_on = True
        active.append(therapy_on)
        crowding = crowding_death(row["total"], config)
        # Update each genotype as one batch.
        offspring = {}
        for genotype, count in population.items():
            # Genotype effects, then therapy kill.
            death, triple = growth_probabilities(genotype, config)
            if crowding:
                death = min(1.0, death + crowding)
            # Therapy acts on natural survivors.
            if therapy_on:
                if genotype & (1 << config.resistance_gene):
                    kill = config.resistant_therapy_kill
                else:
                    kill = config.sensitive_therapy_kill
                death = 1 - (1 - death) * (1 - kill)
            # Survivors, then triple divisions; the rest make two.
            survivors = int(rng.binomial(count, 1 - death))
            triples = int(rng.binomial(survivors, triple))
            # 2 daughters per survivor, +1 per triple.
            n = 2 * survivors + triples
            if n:
                offspring[genotype] = n
        # Mutate after division; effect starts next generation.
        population = mutate_offspring(offspring, config, rng)
    # Return history and stop reason.
    return Result(
        config, seed, history, populations, therapy_generation, stop_reason, active
    )
```

KEY POINTS

- I store the tumor as genotype → count, not as individual cells.
- Two binomial draws per genotype are exactly equivalent to flipping a coin for every cell — just faster.
- Therapy acts on natural survivors, so total death is 1 − (1−d)(1−k).
- Mutations happen in daughters after division; their effect starts next generation.

Mutation: binomial draw per bit, new mutations set with a bitmask OR

simulation.py:184–207

```python
def mutate_offspring(
    population: dict[int, int], config: Config, rng: np.random.Generator
) -> dict[int, int]:
    """Binomial mutation per gene, equivalent to per-cell sampling."""
    # Copy so the caller's dict is untouched.
    population = dict(population)
    # All genes plus the second TSG alleles.
    for gene in range(config.n_genes + len(config.suppressor_genes)):
        bit = 1 << gene
        # Read old, write new: no double counting.
        updated = dict(population)
        for genotype, count in population.items():
            # Mutated genes stay mutated; sample only cells not yet mutated.
            if genotype & bit:
                continue
            # One binomial draw per genotype.
            mutants = int(rng.binomial(count, config.mutation_rate))
            if mutants:
                # Move mutants to genotype | bit.
                updated[genotype] -= mutants
                updated[genotype | bit] = updated.get(genotype | bit, 0) + mutants
        # Drop empty genotypes; total unchanged.
        population = {g: n for g, n in updated.items() if n > 0}
    return population
```

KEY POINTS

- A genotype is a 22-bit integer; mutation is a bitwise OR.
- Mutation only relabels cells — the total count is conserved.

Main result: therapy, nadir, relapse

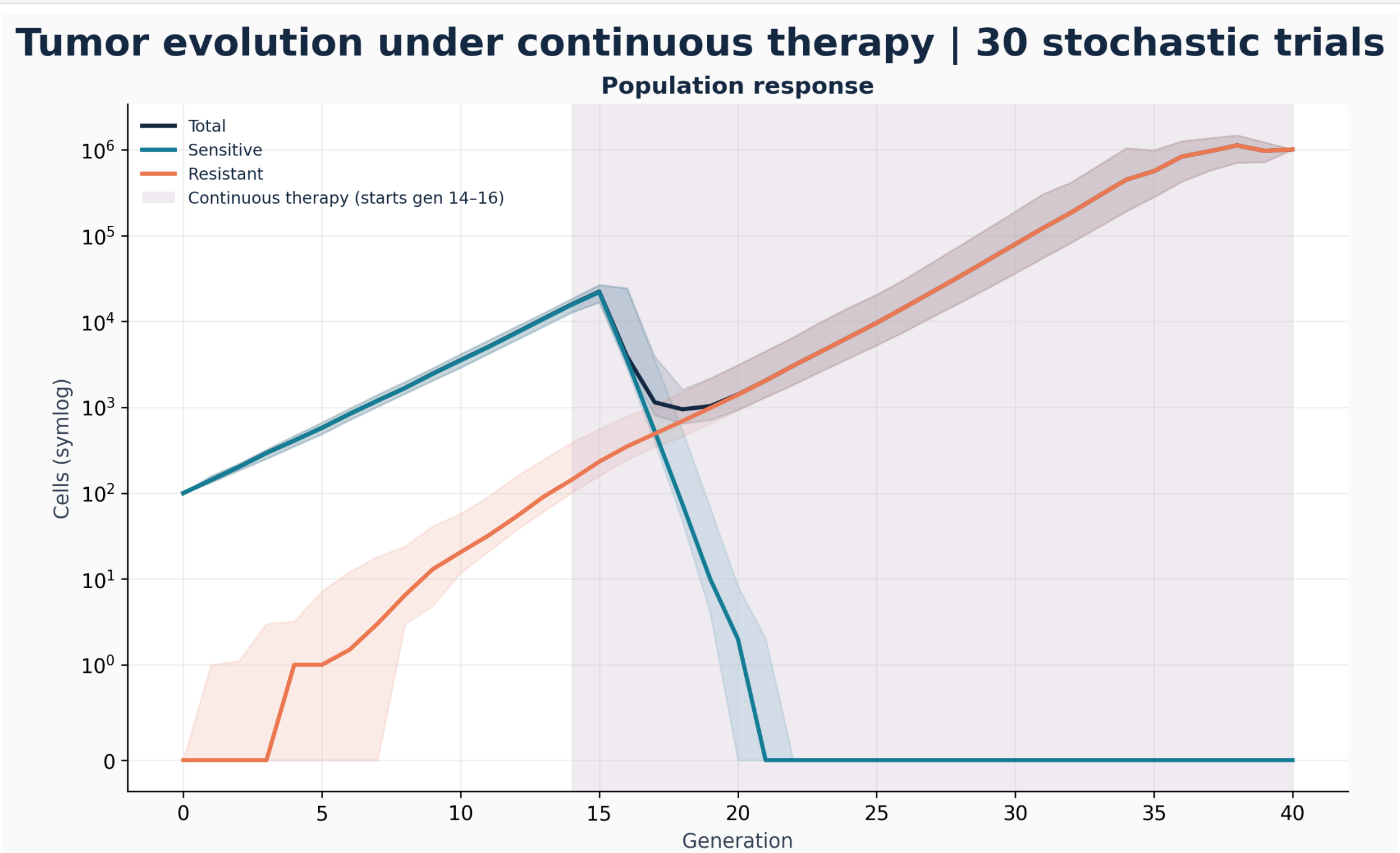
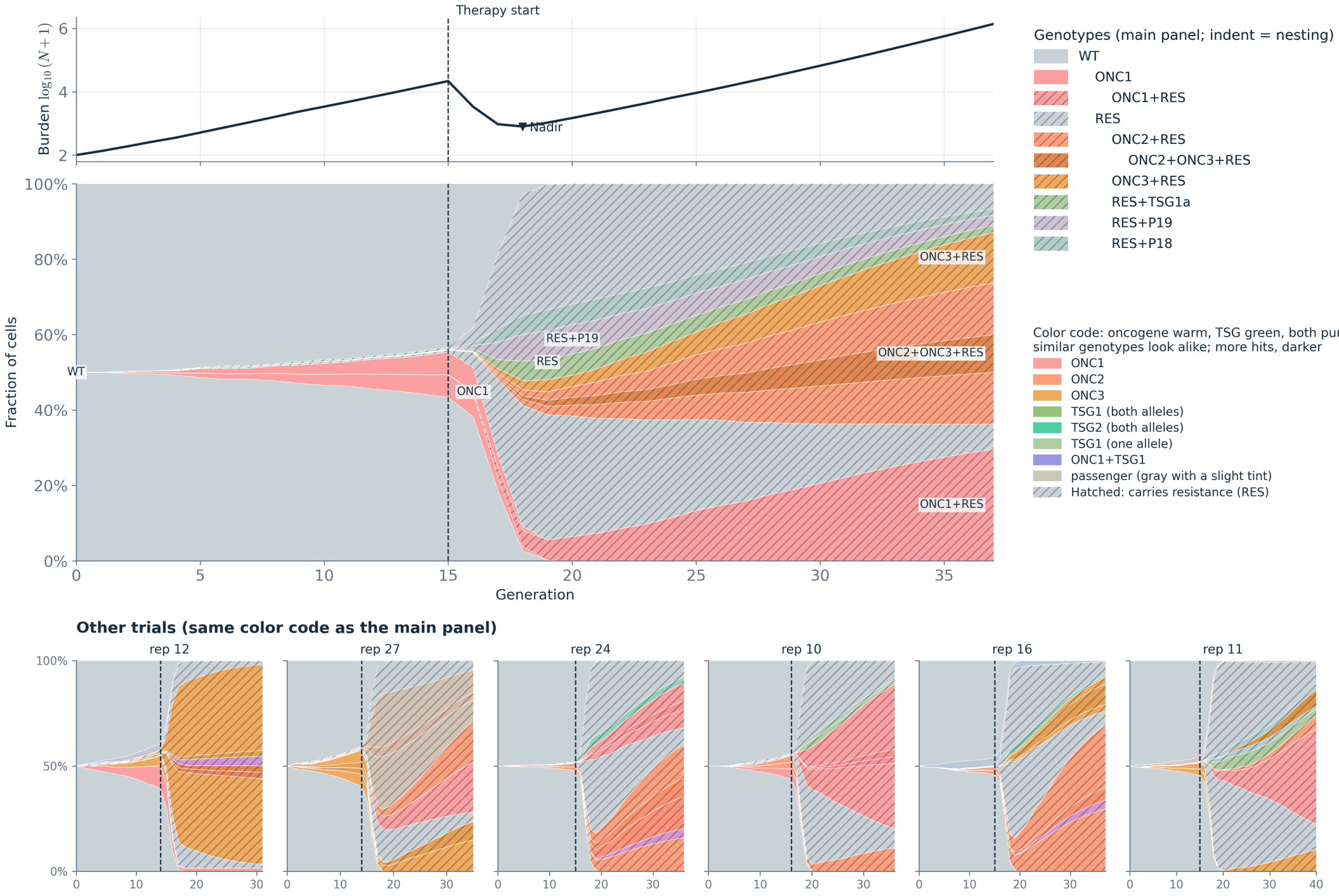


Fig. 01 Tumors under therapy (30 trials)

Genotype composition over time: therapy selects pre-existing resistant genotypes



Other trials (same color code as the main panel)

Muller plot: genotype composition over time

KEY POINTS

- **The winners after therapy are resistant genotypes that also carry an oncogene.**
- **Same outcome in every trial, different genotypes each time — convergent phenotype, divergent genotype.**
- **Passengers hitchhike along with whichever resistant clone happens to win.**

Assignment scenario: one-hit vs two-hit

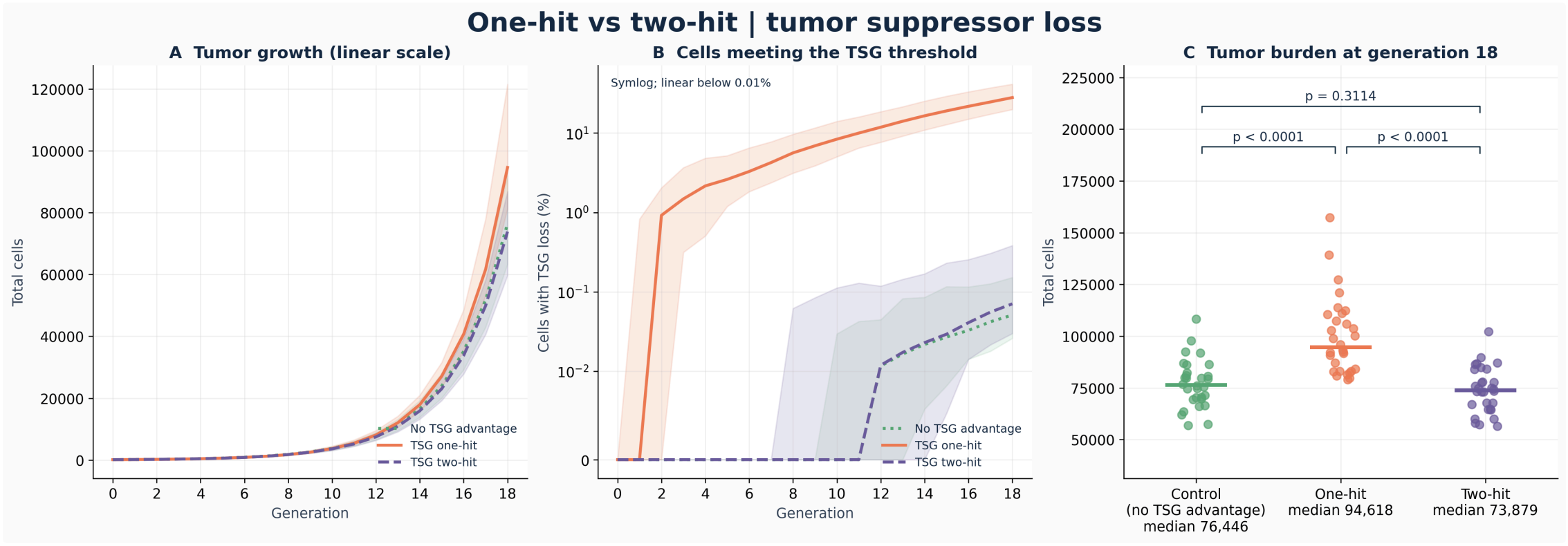


Fig. 03 One-hit vs two-hit

KEY POINTS

- **With one hit, TSG loss spreads within a few generations; with two hits it barely appears in eighteen.**

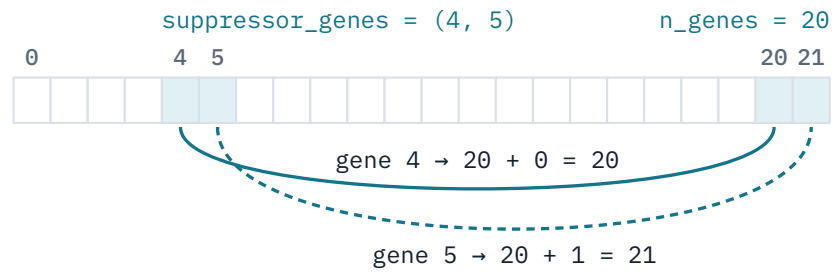
A tumor suppressor is recessive at the cell level: one working allele is enough, so the first hit is silent and gets no selective advantage — it only drifts. The advantage appears only after the second allele of the same gene is lost, roughly μ squared per cell unless the first-hit clone happens to expand first. That waiting time delays two-hit loss: 0.07 percent of cells by generation 18, versus 27.9 percent when one hit is enough.

Two-hit: both allele bits of the same TSG must be lost

simulation.py:134–147

```python
def suppressor_alleles(genotype: int, gene: int, config: Config) -> int:
    """Inactivated alleles of one TSG (0, 1 or 2)."""
    # Second allele bit sits after n_genes.
    second = config.n_genes + config.suppressor_genes.index(gene)
    return int(bool(genotype & (1 << gene))) + int(bool(genotype & (1 << second)))


# Any TSG at its hit threshold?
def suppressor_loss(genotype: int, config: Config) -> bool:
    # Both hits must be in the same gene.
    return any(
        suppressor_alleles(genotype, gene, config) >= config.suppressor_hits
        for gene in config.suppressor_genes
    )
```



second = config.n_genes + config.suppressor_genes.index(gene)

KEY POINTS

The second allele lives in an extra bit beyond the twenty loci, so two different genes with one hit each do not count.

What if the tumor had a higher death rate?

Higher death rate means more turnover: more divisions to reach the same size, so more mutations.

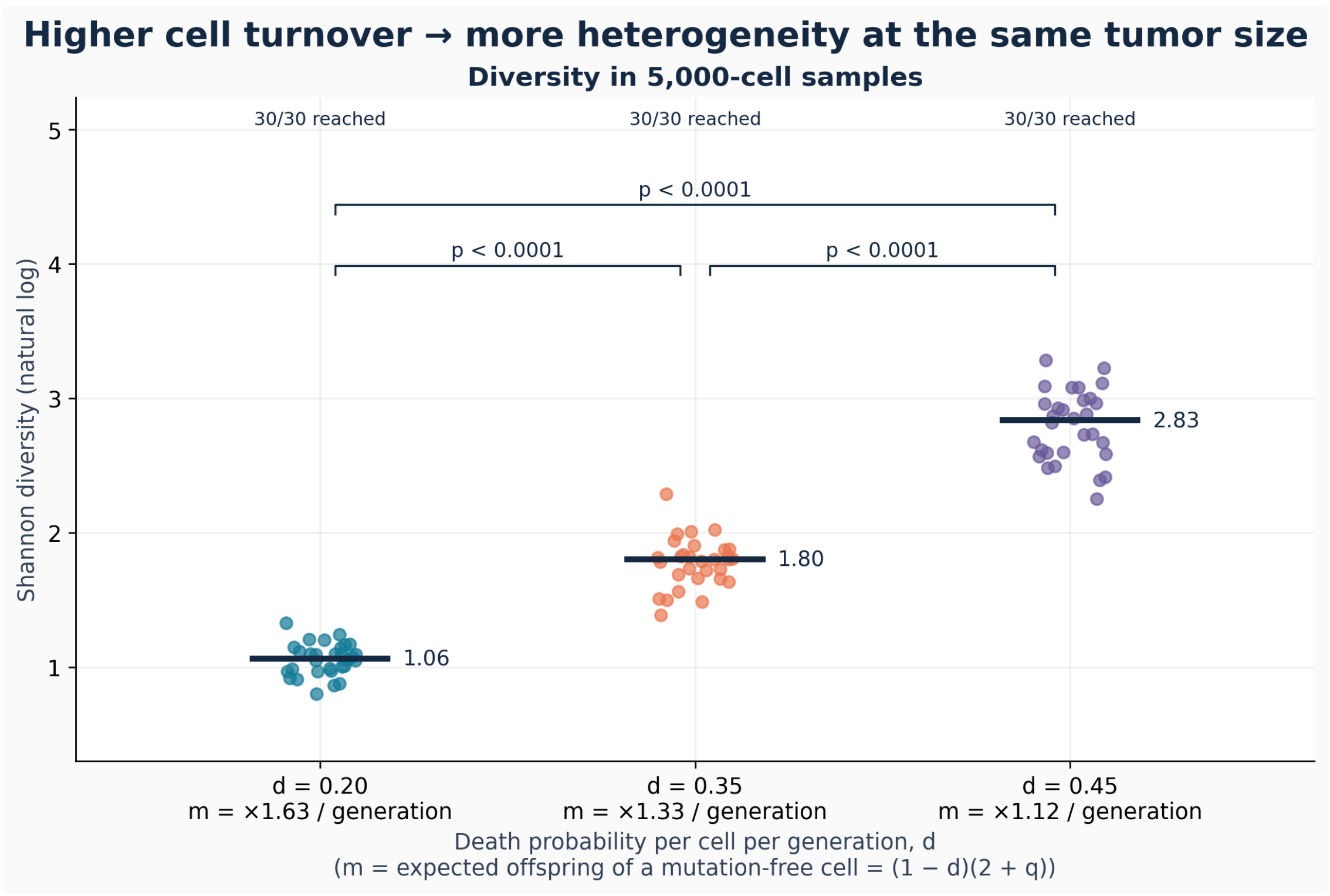


Fig. 02 Death rate and heterogeneity

From description to prediction: theory vs simulation

Goldie–Coldman cure curve

Question: how large can a tumor grow before treatment starts and still be cured completely?

IN ONE SENTENCE

The later therapy starts and the larger the tumor, the more likely it already hides resistant cells; if a single resistant lineage survives, the tumor grows back. So the chance of cure drops steeply with tumor size.

Therapy start (cells)	Simulated cure	Theoretical cure	Surviving resistant lineages Λ
100	99.5%	99.1%	0.009
300	60.5%	54.6%	0.61
500	40.0%	38.8%	0.95
1,000	10.0%	11.9%	2.1
2,000	0.5%	1.1%	4.5
5,000	0%	≈ 0%	9.5

Core idea: cure ⇔ no resistant lineage survives therapy. So compute the expected number of resistant lineages that survive (Λ); the cure probability is the chance that there are none.

NUMBER OF LINEAGES: WHAT DECIDES EXTINCTION

What decides cure is the number of resistant *lineages* that survive therapy, Λ, not the number of resistant cells: if one lineage survives, the tumor grows back. This follows from three model assumptions:

1. Irreversible mutation: resistant cells never revert, so a resistant lineage stays resistant as long as it survives.
2. Independent lineages: one lineage's survival does not affect another's, so each lineage's survival probability can be computed separately and combined.
3. Rare, independent mutational origins: each resistant lineage comes from one independent, rare mutation, so the number of surviving lineages is Poisson distributed.

WHY THE THEORY CURVE IS A STAIRCASE

The tumor does not grow continuously; it jumps ×1.43 per generation. For example, thresholds of 150 and 200 are both first reached in generation 2 (about 204 cells), so therapy actually starts at the same moment and the cure probability is the same; hence the staircase.

VARIABLES IN THE FORMULAS

T: generations of growth before therapy (from 100 cells to the threshold size)
i: generation in which a resistant mutation arises (i = 1, …, T; written s in the code)
S_i: sensitive cells in generation i; S_T: sensitive cells when therapy starts
m: mean offspring per cell without therapy
μ: probability that a daughter cell acquires the resistance mutation
f(z): offspring generating function without therapy; $f^{(n)}$: f applied n times in a row (a lineage after n generations)
f_{on}: offspring generating function of resistant cells on therapy
q_{on}: probability that one resistant lineage eventually dies out on therapy (fixed point of f_{on})
$m_{S,on}$: mean offspring of sensitive cells on therapy (< 1, shrinking each generation)
Λ_{pre}, Λ_{on}: expected numbers of resistant lineages that arise before / during therapy and survive; $\Lambda = \Lambda_{pre} + \Lambda_{on}$
L: number of resistant lineages that actually survive (a random variable); ℓ: a possible value of L, 0, 1, 2, …

KEY MATH

$$q_{\text{on}} = f_{\text{on}}(q_{\text{on}}) = 0.479$$

probability that one resistant lineage eventually dies out on therapy

$$\Lambda_{\text{pre}} = \sum_{i=1}^{T} \underbrace{\mu\, m\, S_{i-1}}_{\substack{A\text{: resistant mutants} \\ \text{arising per generation}}} \times \underbrace{\left[1 - f^{(T-i)}(q_{\text{on}})\right]}_{\substack{B\text{: survival probability} \\ \text{of each resistant lineage}}}$$

expected number of clones that arise before therapy and survive

$$\Lambda_{\text{on}} = \mu\, S_T \underbrace{\frac{m_{S,\text{on}}}{1 - m_{S,\text{on}}}}_{\substack{\text{geometric series sum} \\ = \sum_{j \geq 1} m_{S,\text{on}}^{j}}} (1 - q_{\text{on}})$$

expected number of resistant clones that arise during therapy by mutation of remaining sensitive cells and survive

KEY FUNCTIONS

Poisson cure probability

analysis.py:304–344

```python
def goldie_coldman_theory(config: Config, threshold: float) -> dict:
    """Poisson cure probability: P(cure) ≈ exp(−Λ), baseline cells only."""
    d, q, mu = config.death_rate, config.triple_probability, config.mutation_rate
    off = offspring_law(d, q)
    on_r = offspring_law(1 − (1 − d) * (1 − config.resistant_therapy_kill), q)
    m = (1 − d) * (2 + q)
    m_s_on = m * (1 − config.sensitive_therapy_kill)
    q_on = extinction(on_r)
    T = int(
        np.ceil(
            np.log(max(threshold, config.initial_cells) / config.initial_cells)
            / np.log(m)
            − 1e−12
        )
    )
    S = config.initial_cells * m ** np.arange(T + 1)
    lam_pre, mean_r0, survive = 0.0, 0.0, q_on
    # Go backwards: a clone born at s grows T−s generations.
    for s in range(T, 0, −1):
        births = mu * m * S[s − 1]
        lam_pre += births * (1 − survive)
        mean_r0 += births * m ** (T − s)
        survive = pgf(off, survive)
    lam_on = mu * S[T] * m_s_on / (1 − m_s_on) * (1 − q_on)
    return {
        "threshold": threshold,
        "T": T,
        "N_T": float(S[T]),
        "lambda_pre": lam_pre,
        "lambda_on": lam_on,
        "p_cure": float(np.exp(−(lam_pre + lam_on))),
        "mean_R0": mean_r0,
        "q_on": q_on,
    }
```

With rare, approximately independent mutations, so that the Poisson approximation holds, the number of surviving resistant lineages L follows:

$$L \sim \text{Poisson}(\Lambda), \qquad \Lambda = \Lambda_{\text{pre}} + \Lambda_{\text{on}}$$

The Poisson distribution is:

$$P(L = \ell) = \frac{\Lambda^{\ell} e^{-\Lambda}}{\ell!}$$

and cure requires:

$$L = 0$$

so:

$$\begin{aligned} P(\text{cure}) &= P(L = 0) \\ &= \frac{\Lambda^{0} e^{-\Lambda}}{0!} \\ &= \boxed{e^{-\Lambda}} \end{aligned}$$

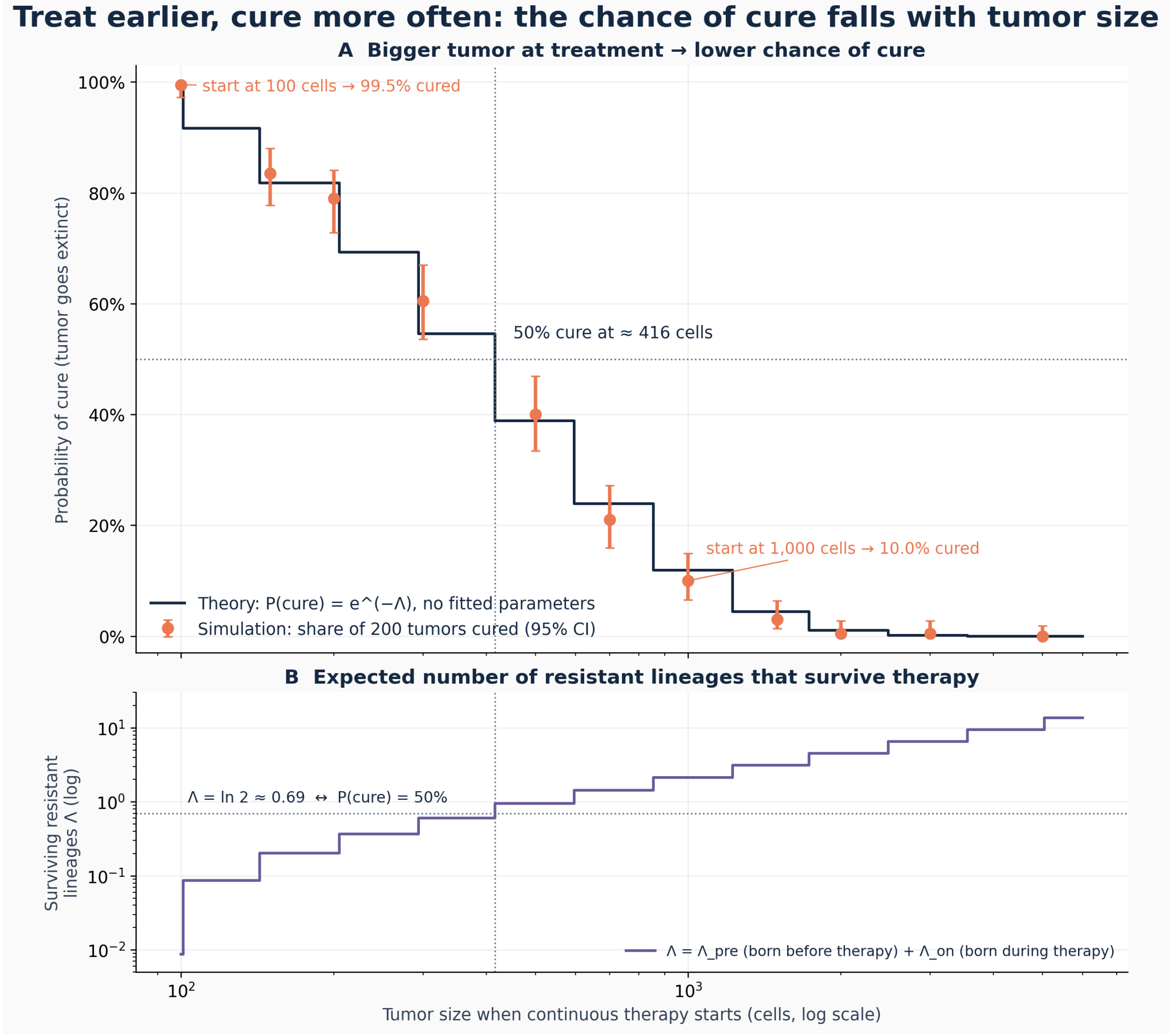


Fig. 08 Goldie–Coldman cure curve

Lineage extinction probability: iterate s ← f(s) from 0 to the smallest fixed point

analysis.py:208–216

```python
def lineage_extinction(death, triple):
    """Lineage extinction: smallest fixed point of f(s)."""
    s = 0.0
    for _ in range(100_000):
        s_next = death + (1 - death) * ((1 - triple) * s**2 + triple * s**3)
        if abs(s_next - s) < 1e-15:
            break
        s = s_next
    return s_next
```

KEY POINTS

The theory reuses the same offspring law as the simulation — iterate s ← f(s) from zero. I iterate rather than use a closed-form solution so the same method still works if the model later allows more than three daughters: for any offspring law, iterating from zero converges to the smallest root.

Limitations and wrap-up

KEY POINTS

- Limitations: no space, no immune system, no quiescent cells, and the mutation rate is inflated for a short demo.
- Bands are ten-to-ninety percent ranges across trials, not confidence intervals, and I did not pick seeds.
- A genotype in my model is not a lineage — the same genotype can arise independently.